Brownout Survival

Eli Keetch

January 2026

1 Overview

Embedded System That Survives Brownouts Without Resetting or Corrupting Data

This project explores how an embedded system can tolerate unstable power conditions by detecting brownouts early, preserving critical system state, entering a safe shutdown, and restoring operation predictably when power returns.

1.1 One-Sentence Summary

This project designs an embedded system that detects power instability, safely saves critical state, and reliably restores operation after a brownout.

2 Problem and Motivation

Most introductory microcontroller projects assume stable power. In real-world systems, voltage drops occur due to weak power supplies, motors starting, loose connections, or unexpected unplugging. During these brownouts, a microcontroller may behave unpredictably before fully resetting, which can lead to corrupted data or unsafe outputs.

This project intentionally explores a more advanced, real-world challenge by designing an embedded system that remains reliable even when power conditions are unstable.

3 Objectives

The system must be able to:

- Detect a brownout before the microcontroller becomes unreliable
- Preserve only critical system state using nonvolatile memory
- Prevent corrupted data from being accepted after power loss
- Enter a safe shutdown state when power is unstable
- Restore valid state or safely revert to defaults on restart

4 System Overview

The system is organized into three major functional blocks.

4.1 Power Awareness

The microcontroller monitors its supply voltage using an ADC connected to a voltage divider (or a comparator if available). When voltage drops below a predefined threshold, the firmware initiates a save-and-shutdown sequence.

4.2 State Preservation

A small snapshot of critical system state is written to nonvolatile memory such as EEPROM, flash, or FRAM. Each snapshot includes a version identifier, sequence number, checksum or CRC, and a valid or commit flag to detect incomplete, or corrupted writes.

4.3 Recovery Logic

On startup, the firmware checks for a valid saved state. If the stored data passes integrity checks, it is restored. If not, the system safely falls back to default values.

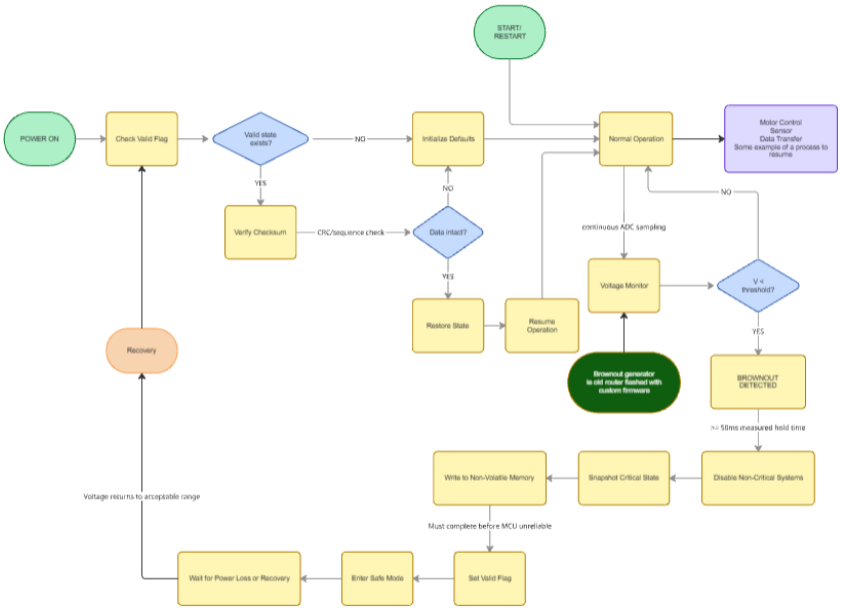


Figure 1: Firmware logic flow for brownout detection, state saving, and recovery.

5 Hardware and Firmware Components

Planned hardware components include a microcontroller board (Arduino, ESP32, or similar), a voltage sensing circuit, bulk capacitance for short hold-up time, nonvolatile storage, and a simple output load such as LEDs, a relay or motor. This is the base goal in applying this to a microcontroller board, but I plan to challenge myself in programming an ATMEGA328P or similar microcontroller directly.

Firmware features include continuous voltage monitoring, configurable brownout thresholds, a state machine for normal operation and shutdown, and structured state packets with integrity checking.

6 Validation and Demonstration

System validation focuses on demonstrating correct behavior under power instability and proving data integrity.

First, a baseline demonstration will be performed with brownout handling disabled to show typical failure behavior. The completed system will then be demonstrated under repeated power interruptions, showing that it preserves state or safely recovers without corruption.

Additional tests include interrupting power during state saves to verify corruption detection, measuring hold-up time provided by bulk capacitance, and performing repeated brownout cycles (25–50) to demonstrate reliability.

7 Deliverables

The final submission will include:

- Block and wiring diagrams
- Fully commented firmware code
- A short report summarizing design choices and measurements
- A live demonstration of brownout survival behavior

8 Stretch Goals

If time and availability permits I would also like to modify this to be an external detector to add onto existing systems, and or serial logging of brownout events.